

Towards Efficient Hardware Implementation of NTT for Kyber on FPGAs

Cong Zhang¹, Dongsheng Liu^{1*}, Xingjie Liu¹, Xuecheng Zou¹, Guangda Niu¹, Bo Liu² and Quming Jiang³

¹National Research Center for Optoelectronics, School of Optical and Electronic Information, Huazhong University of Science and Technology, Wuhan 430074, China

²Key Laboratory of Spintronic Materials, Devices and Systems of Zhejiang, Hangzhou 311300, China

³Chutian Dragon Co., Ltd., Dongguan 523681, China

E-mail: czhanghust@foxmail.com, dslu@hust.edu.cn*

Abstract—Kyber is a promising lattice-based post-quantum cryptography (PQC) for key encapsulation mechanisms. Since number theoretic transform (NTT) is the most computationally expensive operation in Kyber, this paper focuses on novel optimization techniques for efficient hardware implementation of NTT for Kyber. Benefiting from the proposed fast modular multiplication method and a doubled bandwidth ping-pong memory access scheme, our NTT architecture can complete an NTT operation in Kyber in 490 cycles using only 609 LUTs, 640 FFs, 2 DSPs on a Xilinx Artix-7 FPGA. The proposed NTT architecture is 3.95 times faster than the state-of-the-art design for Kyber while achieves an improvement of more than 1.5 times in the area time product (ATP). Compared with the state-of-the-art NTT designs for other algorithms, our NTT architecture reduces 24% FFs and 50% DSPs and ranks second smallest in ATPs, which can also confirm the high efficiency of our design.

Keywords—Kyber, FPGA, number theoretic transform, post-quantum cryptography, hardware efficiency

I. INTRODUCTION

Currently used public-key infrastructures based on RSA and ECC are facing the crucial threat of corruption because of the availability of large-scale quantum computers with Shor's algorithm [1] in the near future. In order to deploy the quantum secure replacements for the cryptographic standards as early as possible, in 2016, the National Institute of Standards and Technology (NIST) announced a post-quantum cryptography (PQC) project [2] to solicit new cryptographic primitives over the world and conduct the standardization process for quantum resistant algorithms. After two rounds of evaluation criteria and selection process, only 7 candidate algorithms are selected forward to the "Third Round Finalists and Candidates" on July 22, 2020 [3]. Among the 7 candidates, the key encapsulation mechanism (KEM) CRYSTALS-KYBER [4], also known as Kyber, which is built on the hardness of the module learning with errors (MLWE) problem, can provide a well trade-off between message sizes and implementation performance while ensuring sufficient resistance against cryptanalytic attacks.

However, very little research has been conducted on implementation of Kyber, especially for the updated version Kyber (v2). In Kyber, NTT is directly used to speed up polynomial modular multiplication, which is the main cost. Thus, whether it is hardware implementation-related research [5] or software implementation-related research on embedded devices [6], [7], they all propose the relevant optimization methods for NTT. For example, in work [5], GS butterfly and dual-column sequential scheme are used in a Kyber processor to improve the performance of NTT operation. Considering

that most lattice-based PQC candidates are NTT-inside algorithms and NTT is the most expensive operation, it is very important to study efficient hardware implementation of NTT.

In this paper, novel optimization techniques for the efficient hardware implementation of NTT for Kyber is proposed and an NTT architecture is implemented on a Xilinx Artix-7 FPGA. The contributions of this paper are as follows:

- A pipeline butterfly arithmetic unit (BAU) based on the fast modular multiplication method is proposed with resources and critical path well optimized. The BAU-based NTT architecture reduces 24% FFs and 50% DSPs compared with related NTT designs and runs at a maximum operating frequency of 257MHz.
- A doubled bandwidth ping-pong memory access scheme that can eliminate the bottleneck of RAM access is presented. Applying the technique, our NTT architecture can complete the NTT operation in Kyber 3.95 times faster than the state-of-the-art design for Kyber.

The rest of the paper is organized as follows. The preliminaries of Kyber and NTT are introduced in Section II. Section III presents the optimization techniques and architecture of NTT. Section IV shows comparisons of resource and performance results on the FPGA platform with related works. Finally, this paper is concluded in Section V.

II. PRELIMINARIES

A. Notation

The quotient ring of integer polynomials is represented as $R_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$, in where each coefficient of the n -degree polynomial is reduced modulo q . The polynomials in R_q are all expressed in bold letters throughout the paper, and a bold letter with a hat corresponds to its representation after performing an NTT operation.

B. Kyber

Kyber, as described in [4], is built on the hardness of the Module-LWE (MLWE) problem, it uses a $k \times k$ matrix of polynomials in R_q as the public information $\mathbf{A}$, whereas $\mathbf{s}$ and $\mathbf{e}$ become k -dimensional vectors of polynomials. Specifically, Kyber (v2) uses $q=3329$ ($q=7681$ in the Kyber (v1)) and $R_q = \mathbb{Z}_{3329}[x]/\langle x^{256} + 1 \rangle$ for all security levels (Kyber-512, Kyber-768, Kyber-1024 which use $k=2, 3, 4$, respectively). In Kyber.CPA-secure encryption scheme, one of the main computations is $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$, where the public key is $(\mathbf{A}, \mathbf{t})$ and the private key is $\mathbf{s}$. The main cost in Kyber schemes is

arithmetic in R_q and even more specifically polynomial multiplications. The main cost of polynomial multiplications are the (forward and inverse) NTT operations [6].

C. NTT

The NTT [8] is a tool widely used in the implementations of lattice-based cryptography to speed up polynomial multiplication. The NTT operation on a polynomial is actually substituting some special values into the polynomial, transforming the polynomial from the coefficient representation to the point value representation. NTT uses the n -th primitive root of unity ω_n in the ring $\mathbb{Z}_q$ where $\omega_n^n = 1 \bmod q$ and $\omega_n^i \neq 1 \bmod q$ for $i \neq n$. For the polynomials in R_q , $\mathbf{a}$ with n coefficients ($a_0, a_1, \dots, a_{n-1}$) has its NTT representation $\hat{\mathbf{a}} = \text{NTT}(\mathbf{a})$ with the n -point values ($\hat{a}_0, \hat{a}_1, \dots, \hat{a}_{n-1}$), where

$$\hat{a}_k = \sum_{i=0}^{n-1} a_i \omega_n^{ik} \bmod q, \quad k = 0, 1, \dots, n-1. \quad (1)$$

Note that the inverse NTT (INTT) operation is similar to NTT, besides that the ω_n is replaced by ω_n^{-1} and an additional multiplication with n^{-1} is required. The INTT operation converts $\hat{\mathbf{a}}$ back to $\mathbf{a}$ can be defined as follows:

$$a_i = n^{-1} \sum_{k=0}^{n-1} \hat{a}_k \omega_n^{-ik} \bmod q, \quad i = 0, 1, \dots, n-1. \quad (2)$$

With the application of NTT and INTT, the multiplication of two polynomials $\mathbf{a}$ and $\mathbf{b}$ can be computed as $\mathbf{c} = \text{INTT}(\text{NTT}(\mathbf{a}) \circ \text{NTT}(\mathbf{b}))$, where $\circ$ is the point-wise multiplication (PWM). The time complexity of n -point

Algorithm 1 Iterative NTT operation inside Kyber

Input: Polynomial $\mathbf{a} \in R_q$ and 256-th primitive root of unity $w' \in \mathbb{Z}_q$
Output: $\hat{\mathbf{a}} = \text{NTT}(\mathbf{a})$

```

1 begin
2    $\mathbf{a} \leftarrow \text{Bit Reverse}(\mathbf{a})$ 
3   for  $m = 2$  to 128 by  $m = 2m$  do
4      $\omega'_m \leftarrow \omega'^{128/m}$ 
5      $\omega \leftarrow 1$ 
6     for  $j = 0$  to  $m/2 - 1$  do
7       for  $k = 0$  to 127 by  $m$  do
8          $t_H \leftarrow \omega \cdot \hat{a}_{2(k+j+m/2)+1}$ ,  $t_L \leftarrow \omega \cdot \hat{a}_{2(k+j+m/2)}$ 
9          $u_H \leftarrow \hat{a}_{2(k+j)+1}$ ,  $u_L \leftarrow \hat{a}_{2(k+j)}$ 
10         $\hat{a}_{2(k+j)+1} \leftarrow u_H + t_H$ ,  $\hat{a}_{2(k+j)} \leftarrow u_L + t_L$ 
11         $\hat{a}_{2(k+j+m/2)+1} \leftarrow u_H - t_H$ ,  $\hat{a}_{2(k+j+m/2)} \leftarrow u_L - t_L$ 
12      end
13       $\omega \leftarrow \omega \cdot \omega'_m$ 
14    end
15  end
16 end
```

polynomial multiplication can be decreased from $O(n^2)$ for using schoolbook multiplication to $O(n \log n)$ for using NTT technology. In Kyber (v2), $n=256$ and $q=3329$ is chosen such that $\mathbb{Z}_q$ contains 256-th roots of unity, but not 512-th roots of unity [4]. As a consequence, the NTT of a polynomial $\mathbf{a}$ is a vector of 128 polynomials of degree at most 1 with 2 coefficients each: $\hat{\mathbf{a}} = (\hat{a}_0 + \hat{a}_1 x, \hat{a}_2 + \hat{a}_3 x, \dots, \hat{a}_{254} + \hat{a}_{255} x)$

with $\hat{a}_{2i} = \sum_{j=0}^{127} a_{2j} \omega'^j$, $\hat{a}_{2i+1} = \sum_{j=0}^{127} a_{2j+1} \omega'^j$, $\omega' = \omega_{256}^{2\text{br}(i)+1}$,

where $\text{br}(i)$ means bit-reverse order of i . Algorithm 1 shows the detailed steps of iterative in-place NTT operation in Kyber.

III. HARDWARE IMPLEMENTATION

A. BAU

To accelerate the NTT/INTT and PWM operation on polynomials, this paper proposes an efficient pipelined-structure called butterfly arithmetic unit (BAU) that can support continuous butterfly operations and modular multiplications. The detailed structure of BAU is shown in Fig.1 (a). The BAU has four 12-bit input ports a, b, d, ω and three 12-bit output ports $a+p, a-p, p$ for the modulo q is 12-bit (3329). When an NTT/INTT operation is performed, the port $a+p$ outputs $a+b \cdot \omega \bmod q$ and port $a-p$ outputs $a-b \cdot \omega \bmod q$. When executing a PWM operation, the port p outputs $b \cdot \omega \bmod q$ and port $a+p$ outputs $d+b \cdot \omega \bmod q$.

The high efficiency of the BAU mainly benefits from the proposed fast modular multiplication method below. By gradually calculating some intermediate values o, h, m, l, s, j and k , the modular multiplication $p \equiv b \cdot \omega \bmod q$ is finally transformed into the modular addition $p \equiv j + k \bmod q$. For the values h, m and l , they are the parts of the 24-bit product o reduced modulo q . As there is no such an efficient way to solve an operation like $o[23:20] \cdot 2^{20} \bmod q$ through arithmetic logic units, three fast look-up tables that respectively store the precomputed values of $o[23:20] \cdot 2^{20} \bmod q$, $o[19:16] \cdot 2^{16} \bmod q$ and $o[15:12] \cdot 2^{12} \bmod q$ are used to directly get h, m and l in the BAU. The inputs of the three fast look-up tables are all 4-bit and their outputs are all 12-bit. When $o[23:20]$, $o[19:16]$ and $o[15:12]$ are used as the input addresses, the outputs of three fast look-up tables are corresponding h, m, l . For the value s , it's a sum of four 12-bit number and can be calculated through two 12-bit adder followed by one 13-bit adder. For the value j , it has the same form as h, m, l , but it can be calculated through a 4 to 1 MUX with four precomputed values of $s[13:12] \cdot 2^{12} \bmod q$ according to $s[13:12]$. For the value k , it equals to the reduction in modulo q of $s[11:0]$, so a 12-bit subtractor and a 2 to 1 MUX are adopted to output $s[11:0]$ or $s[11:0]-q$.

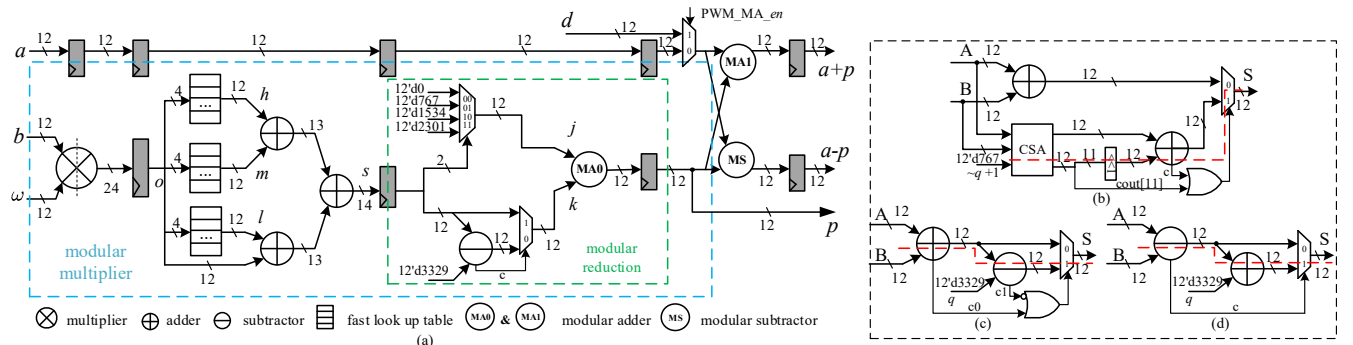


Fig. 1. (a) The detailed structure of BAU. (b) Type-0 modular adder (MA0). (c) Type-1 modular adder (MA1). (d) Modular subtractor (MS).

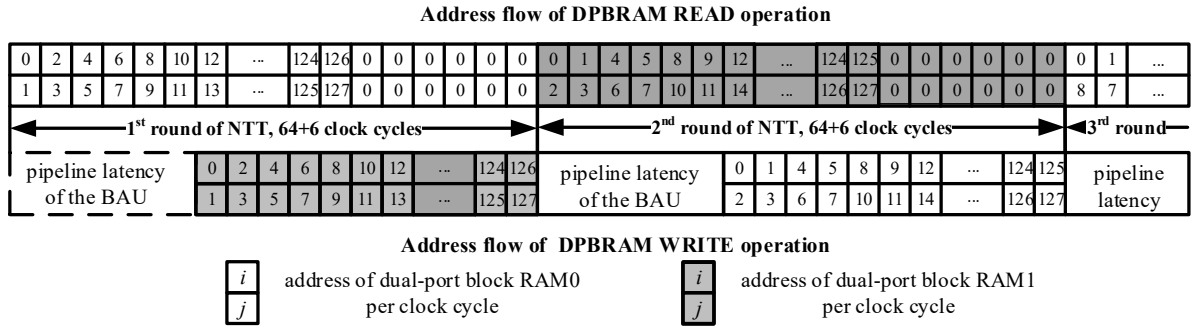


Fig. 2. The address flow of RAM0 and RAM1 when an NTT/INTT operation is performed.

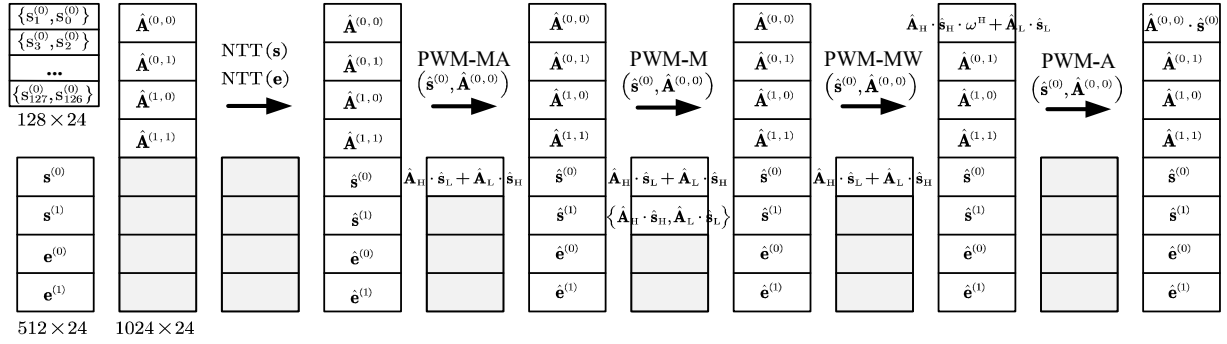


Fig. 3. Part of the data flow for Kyber-512 in memory.

The BAU is implemented with a 5-stage pipeline structure. To further reduce the critical path delay, the type-0 modular adder given in Fig.1 (b) is used at the fourth pipeline stage. Benefits from the carry save adder (CSA) with a negligible logic delay (actually equals the delay of three logic gates), the type-0 modular adder has a logic delay $d_{\text{MA0}} \approx d_{\text{Adder}} + d_{\text{MUX}}$ and the path delay of the fourth pipeline stage is reduced to slightly larger than $2d_{\text{Adder}} + 2d_{\text{MUX}}$. Then the timing path at the third pipeline stage becomes the critical path of BAU, which is only $d_{\text{LUT}} + 2d_{\text{Adder}}$. Considering the type-1 modular adder consumes less logic resources than the type-0 one and the path delay of the last pipeline stage is less than the third stage, the type-1 modular adder is still used at the last pipeline stage. Overall, the critical path delay of our BAU is less than the logic delay of three 12-bit adders.

B. Ping-Pong Memory Access

An NTT operation consists of consecutive butterfly operation that needs to read two coefficients from the memory and write two calculation results back. Even if it is dual-port block RAM (DPBRAM) on FPGAs, a read or write operation of two data takes at least 1 clock cycle. Thus, it always takes 2 clock cycles to complete one butterfly operation. Considering the BAU works in pipeline mode, it can read and write two data every clock cycle when the pipeline is fulfilled. To eliminate the bottleneck of memory access and avoid read-write conflicts, this paper proposes a ping-pong memory access scheme which enables reading and writing two coefficients in 1 cycle by using BAU interacting with two DPBRAMs (indicated by RAM0 and RAM1).

When performing an NTT/INTT operation on a 128-degree polynomial in RAM0, the addresses flows of RAM0 and RAM1 for read operation and write operation in every clock cycle are given in Fig.2. Every for-loop in line 3 of Algorithm 1 is a round that consists 64 butterfly operations, RAM0 and RAM1 work in read or write mode in turn in each round of NTT operation. At the first round of an NTT

operation, RAM0 is in read mode, RAM1 is in write mode. In every clock cycle, two data are fetched from RAM0 and sent to the input port a and b of BAU respectively, the corresponding twiddle factor would also be fetched and sent to port ω . When two data at the output port $a+p$ and $a-p$ of BAU are updated, the two new data are written into the corresponding addresses of RAM1. When the last two data of 128-degree polynomial in RAM0 are read out, 128 intermediate values would be all stored in RAM1 accordingly after the pipeline latency of BAU (6 clock cycles), then the first round of NTT is completed. At the next round of NTT, RAM0 changes to write mode, RAM1 changes to read mode, and the second 128 intermediate values would be written into RAM0. It can be seen that the read-object and write-object are exchanged when the new round begins. The NTT operation in Kyber-512 needs 7 ($\log_2 128$) rounds, and the final result of an NTT operation would be stored in RAM1. As there is no address conflict due to the different read and write objects, therefore, the ping-pong memory access scheme avoids the bit-reverse order step before an NTT operation, which can save 256 ($n=256$) clock cycles.

In addition, the word size of the two DPBRAM is set as 24 ($2\log 3329$), two coefficients from the odd ($2i+1$) and even ($2i$) parts of a 256-degree polynomial are stored in pairs in one memory location, i.e. $\{s_1^{(0)}, s_0^{(0)}\}$ and $\{s_{127}^{(0)}, s_{126}^{(0)}\}$, etc. Then two BAUs are applied to parallelly deal with two butterfly operations or two modular multiplications on two pairs of coefficients to reduce clock cycles. Specifically, through doubling the bandwidth, one NTT operation in Kyber-512 would take only 490 cycles ($((64+6) \cdot \log_2 128)$). For the PWM operation, it is divided into 4 steps and the corresponding calculation of each step is mainly completed through the interaction of port d and port p between the two BAUs, totally taking only 536 cycles ($4 \cdot (128+6)$). Part of the data flow for Kyber-512 is shown in Fig.3, RAM0 is a 18Kb DPBRAM configured as 512×32 that can store 4 sets of polynomials and RAM1 is a 36Kb one configured as 1024×32 that can store 8

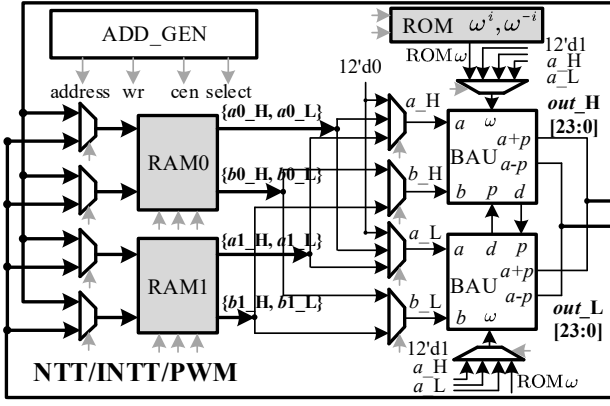


Fig. 4. The architecture of NTT for Kyber-512.

sets of polynomials, $\hat{s}_L$ and $\hat{s}_H$ respectively represent the even part and odd part of $\hat{s}^{(0)}$ (one set of polynomial in $\hat{s}$).

C. Architecture of NTT

The architecture of the presented NTT for Kyber-512 ($n=256$, $q=3329$, $k=2$) is depicted in Fig.4. The NTT mainly consists of two BAUs, two DPBRAMs, a 256×12 block ROM (BROM) and a control unit ADD_GEN. The BROM is used to store precomputed twiddle factors for NTT/INTT operation. The control unit ADD_GEN generates addresses, write/read and enable signals for DPBRAM and BROM, and provides the corresponding select signals for NTT or INTT or PWM operation.

In order to better compare with the related state-of-the-art NTT design for NewHope schemes [9]–[13], the presented NTT architecture is also configured to support the NTT operation in NewHope-1024 ($n=1024$, $q=12289$, $k=1$). Simpler control logic, one BAU (14-bit), two 1024×14 DPBRAMs and a 3072×14 BROM are applied.

IV. IMPLEMENTATION RESULTS AND COMPARISON

For a fair comparison with the state-of-the-art related hardware implementations, this paper implements the NTT architecture on the same Xilinx Artix-7 FPGA platform with the part number XC7A35TFTG256-1. The implementation results are obtained from the Xilinx ISE 14.7 design suite with all options of synthesis and post place-and-route (Post-PAR) being optimized for area-performance balance. Learning from the utilization report after Post-PAR, our implementation totally takes 609 LUTs, 640 FFs, 2 DSPs and 4 18Kb BRAMs for Kyber-512 and 465 LUTs, 286 FFs, 2 DSPs and 5 18Kb

BRAMs for NewHope-1024. In Table I, detailed comparison on hardware implementation results of NTT designs for Kyber-512 and NewHope-1024 are listed.

In work [5], a vector of polynomial processor for Kyber is given, the processor cuts off 29.4% clock cycles for Kyber-512 by applying Gentlemen-Sande butterfly and a dual-column sequential scheme, utilizing only 477 LUTs, 237 FFs and 1 DSP. Though our NTT architecture is $1.5 \times$ larger than the processor in [5], it can complete the NTT operation in Kyber-512 $3.95 \times$ faster than [5] due to the efficient BAU and doubled bandwidth ping-pong memory access scheme. Totally, the ATPs of the processor in [5] measured by LUTs, FFs, DSP, BRAM are $2.9 \times$, $1.5 \times$, $1.9 \times$, $2.9 \times$ than our NTT, demonstrating that our NTT architecture has much higher efficiency. Compared to the memory-efficient high-speed implementation of Kyber on an ARM Cortex-M4 microcontroller in [6], our NTT architecture cuts off nearly 93.6% clock cycles for the NTT operation in Kyber-512 and can run $169 \times$ faster. This also shows that specific hardware implementation has much better performance than software implementation.

Comparisons with state-of-the-art NTT designs for NewHope-1024 are also conducted. The NTT structure proposed in [12] has the smallest ATPs measured by FF, DSP and BRAM among the NTT designs. Its advantages mainly come from the use of optimized address generation method. But the ATPs of our NTT architecture measured by LUTs, FFs, DSP, BRAM are only $1.2 \times$, $1.7 \times$, $1.1 \times$ and $1.1 \times$ than it, ranking the second. And our NTT architecture consumes 55% LUTs, 76% FFs and 50% DSPs of it. Compared with the configurable NTT module proposed in [13], our NTT architecture consumes 135% LUTs, 58% FFs and 33% DSPs of it. Totally, our BAU-based NTT reduces more than 24% FFs and 50% DSPs and ranks the second smallest in ATPs.

V. CONCLUSION

In this work, elaborate optimization techniques are proposed for the efficient hardware implementation of NTT for Kyber. Taking advantages of the optimizations including pipeline BAU and doubled bandwidth ping-pong memory access scheme, the presented NTT architecture can complete the NTT operation in Kyber-512 $3.95 \times$ faster than a vector of polynomial processor for Kyber and achieves at least $1.5 \times$ improvement in ATPs. As for comparisons with the state-of-the-art NTT designs, the NTT architecture has the minimal hardware resource overhead and the second smallest ATPs when configured to support the parameters for NewHope

TABLE I. COMPARISON ON IMPLEMENTATION RESULTS OF NTT/INTT

Design	Scheme	Parameter ($n/q/k$)	Max Freq. (MHz)	NTT/INTT Clock Cycles	Time (μ s)	LUTs /ATP ^a	FFs /ATP	DSPs /ATP	BRAMs ^b /ATP
This work	Kyber-512(v2)	256/3329/2	257	490/490	1.9	609/1.1	640/1.2	2/3.8	4 ^b /7.6
[5]	Kyber-512(v1)	256/7681/2	136	1024/1024	7.5	442/3.3	237/1.8	1/7.5	3 ^b /22.5
[6]	Kyber-512(v2)	256/3329/2	24	7725/9347	321.9	—	—	—	—
This work	NewHope-Simple	1024/12289/1	225	5180/6211	23.0	465/10.7	286/6.6	1/23.0	5/115.0
[9]	NewHope-Simple	1024/12289/1	125	35845/35845	286.7	1390/398.5	615/176.3	2/573.2	1/286.7
[10]	NewHope	1024/12289/1	152	1280/1280 ^c	8.4	2696 ^d /22.6	2982/25.0	8/67.2	— ^d
[11]	NewHope	1024/12289/1	150	1592/1592 ^c	10.6	2832/30.0	1381/14.6	8/84.8	10/106.0
[12]	NewHope-NIST	1024/12289/1	244	2569/2569	10.5	847/8.9	375/3.9	2/21.1	6/63.2
[13]	NewHope-NIST	1024/12289/1	251	6206/6206	24.7	343/8.5	493/12.2	3/74.2	6/148.4

^a The area-time product (ATP) is calculated by multiplying the number of consumed resources by the time required to execute the NTT/INTT operation. The smaller the ATP, the higher efficiency of the corresponding design.

^b The number of equivalent 18Kb BRAMs.

^c The cycles for order reverse are not included.

^d The distributed RAM and ROM instead of BRAMs are applied in [10]. Only the LUTs occupied by the butterfly unit and reduction unit are taken into account for comparison.

schemes. According to the implementation results on FPGAs, the presented NTT architecture has advantages in both speed and ATPs, it is well suitable for Kyber and other NTT-inside lattice-based PQC algorithms.

ACKNOWLEDGEMENT

This work is supported by the National Natural Science Foundation of China (No. 61874163), the National Science and Technology Major Project (No. 2017ZX01032-101), the Dongguan Innovative Research Team Program (No. 201760712600139), and the Fundamental Research Funds for the Central Universities (HUST: 2018KFYYXJJ056).

REFERENCES

- [1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [2] D. Moody, "Post-quantum cryptography: NIST's plan for the future," in *Proc. 16th Conf. Talk Given PQCrypto*, Fukuoka, Japan, Feb. 2016, pp. 23–26. [Online]. Available: <https://pqcrypto2016.jp/data/pqc2016/nistannouncement.pdf>.
- [3] C. Boutin, "NIST's Post-Quantum Cryptography Program Enters 'Selection Round'," *News of NIST*. [Online]. Available: <https://www.nist.gov/news-events/news/2020/07/nists-post-quantum-cryptography-program-enters-selection-round>.
- [4] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber (version 2.0) – Submission to round 2 of the NIST post-quantum project," *Resources of CRYSTALS*. [Online]. Available: <https://pq-crystals.org/kyber/data/kyber-specification-round2.pdf>.
- [5] Z. Chen, Y. Ma, T. Chen, J. Lin and J. Jing, "Towards Efficient Kyber on FPGAs: A Processor for Vector of Polynomials," *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, Beijing, China, 2020, pp. 247–252.
- [6] L. Botros, M. J. Kannwischer, P. Schwabe, "Memory-Efficient High-Speed Implementation of Kyber on Cortex-M4," in *10th AFRICACRYPT 2019*, Rabat, Morocco, 2019, pp.209–228.
- [7] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "ARM Cortex-M4 optimized implementation of Kyber," *Resources of CRYSTALS*. [Online]. Available: <https://github.com/pq-crystals/kyber>.
- [8] D. J. Bernstein, "Fast Multiplication and its Applications," *Algorithmic Number Theory*, 2008, vol. 44, pp. 325–384.
- [9] T. Oder and T. Guneyasu, "Implementing the newhope-simple key exchange on low-cost fpgas," in *Progress Cryptology—LATINCRYPT*, T. Lange and O. Dunkelman, Eds. Cham, Switzerland: Springer, 2019, pp. 128–142.
- [10] Y. Xing and S. Li, "An Efficient Implementation of the NewHope Key Exchange on FPGAs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, March 2020, vol.67, no.3, pp. 866–878.
- [11] P.-C. Kuo, W.-D. Li, Y.-W. Chen, Y.-C. Hsu, B.-Y. Peng, C.-M. Cheng, and B.-Y. Yang, "High performance post-quantum key exchange on fpgas," *Cryptol. ePrint Arch., Tech. Rep.* 2017/690, 2017. [Online]. Available: <https://eprint.iacr.org/2017/690>.
- [12] N. Zhang, B. Yang, C. Chen, S. Yin, S. Wei, and L. Liu, "Highly Efficient Architecture of NewHope-NIST on FPGA using Low-Complexity NTT/INTT," in *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020, vol.2020, No.2, pp.49–72.
- [13] A. Jati, N. Gupta, A. Chattopadhyay, and S. K. Sanadhya, "Spqcop: Side-channel protected post-quantum cryptoprocessor," *Cryptology ePrint Arch., Tech. Rep.* 2019/765, 2019. [Online]. Available: <https://eprint.iacr.org/2019/765>.